

layers to add

Exactly. You're now thinking like an infrastructure guy instead of a trader.

Your original idea was:

```
Binance
|
Exchange API
|
Freqtrade Engine
```

Monitor:

- Is Binance reachable?
- Is CCXT/API connected?
- Are candles arriving?
- Are orders being submitted?
- Are orders being filled?
- Is wallet data updating?

That's already valuable.

But after setting up Azure tonight, you discovered there are actually **two lower layers** that can fail before the bot even reaches Binance.

```
Layer 5: Strategy
Layer 4: Freqtrade Engine
Layer 3: Exchange API / Binance
Layer 2: Docker
Layer 1: VPS / VM
```

Layer 1 - VPS / VM

Things that can break:

- VM powered off

- Azure suspended
- VPS rebooted
- Disk full
- CPU stuck at 100%
- RAM exhausted
- Network disconnected

Checks:

- ping VM
 - SSH reachable?
 - CPU %
 - RAM %
 - disk %
 - uptime
-

Layer 2 - Docker

Things that can break:

- container stopped
- container restarting endlessly
- image corruption
- volume issues

Checks:

```
docker ps
```

and

```
docker inspect
```

You can know:

- running
 - exited
 - restarting
 - unhealthy
-

Layer 3 - Exchange Connectivity

Things that can break:

- Binance API outage
- API key revoked
- invalid permissions
- rate limits
- clock drift

Checks:

```
exchange.fetch_balance()
```

or

```
exchange.fetch_ticker()
```

If that fails:

```
ALERT:  
Binance API unreachable
```

Layer 4 - Freqtrade Engine

Things that can break:

- bot frozen

- process crashed
- strategy exception
- heartbeat stopped

Checks:

```
docker logs freqtrade
```

or

REST API:

```
/api/v1/ping
```

or

```
/api/v1/status
```

Layer 5 - Trading Logic

Things that can break:

- no trades for 10 days
- pairlist empty
- strategy disabled
- stuck position
- abnormal losses

Checks:

```
/api/v1/trades
```

```
/api/v1/performance
```

```
/api/v1/status
```

This is where your idea becomes interesting:

Most Freqtrade users already monitor **Layer 3, 4 and 5**.

Almost nobody monitors **all 5 layers together**.

Imagine a dashboard:

VM	✓
Docker	✓
Freqtrade	✓
Binance	✗
Strategy	⚠

or

VM	✓
Docker	✗
Freqtrade	✗
Binance	?
Strategy	?

The second case immediately tells the user:

"Don't waste time checking Binance.

The container died 3 hours ago."

That's operational intelligence, not just bot monitoring.

And the funny part is: tonight's Azure setup already gave you real examples of Layer 1 and Layer 2 failures:

- Port 8080 closed in Azure NSG → Layer 1 networking issue.
- Docker binding to `127.0.0.1` → Layer 2/container issue.

- Freqtrade itself was healthy the whole time.

Those are exactly the incidents a monitoring product should detect automatically.

Yes, but I'd separate "**strategy monitoring**" from "**system monitoring**" because they're fundamentally different.

The first 5 layers are objective:

```
Layer 1: VM/VPS
Layer 2: Docker
Layer 3: Exchange API
Layer 4: Freqtrade Engine
Layer 5: Execution / Orders
```

You can say with certainty:

```
Docker running? YES
Docker running? NO

Binance reachable? YES
Binance reachable? NO
```

No ambiguity.

Strategy monitoring is different.

Suppose a user's strategy intentionally trades only once every 2 weeks.

If your watchdog sends:

```
ALERT:
```

No trade for 5 days.

That's a false alarm.

For another trader:

No trade for 5 days.

might actually indicate a broken strategy.

So strategy monitoring becomes:

Layer 6: Behavioral Monitoring

Examples:

- No trades for X days
- Win rate collapsed
- Profit factor collapsed
- Pairlist empty
- Position stuck for too long
- Drawdown exceeded threshold
- Unexpected number of signals

These are configurable.

Then there's another layer most people miss:

Layer 7: Notification System

Imagine everything breaks and your monitoring system detects it.

But Telegram is down.

Or the webhook fails.

Or email fails.

Then the alert never reaches the user.

So even monitoring platforms need monitoring.

Example:

Layer 1 VM	✓
Layer 2 Docker	✓
Layer 3 Binance	✗
Layer 4 Freqtrade	✓
Layer 5 Orders	△
Layer 6 Strategy	△
Layer 7 Alerts	✓

For your SaaS, I'd think about it like this:

Core Offer (what every customer gets)

VM/VPS monitoring
Docker monitoring
Freqtrade monitoring
Exchange monitoring
Order execution monitoring

This is the operational reliability layer.

Pro Offer

Add:

Strategy health
Performance analytics
Drawdown alerts
Behavior anomalies

This is where things become trader-specific.

The reason I separate them is that Layers 1-5 are universal. Every Freqtrade user needs them.

Layer 6 depends on the strategy and becomes much harder to support because every trader defines "healthy" differently.

That's why I'd build Layers 1-5 first. They are cleaner, easier to automate, and closer to the operational pain you've personally experienced while setting up and running bots.

script:

ok i got u but well my code tells another story haha , like spread *8 so this is layer 3 an issue about the market not the system , that's layer3 i did that , and for layer2 is for latency and if bot is alive that's it but it turns out it's much richer than i thought i should add those docker etc... it should be a full system on operations elvel , so layer1 will be operations (docker monitoring , vm/vps, engine monitoring , exchange monitoring) , layer2 market monitoring(order execution monitoring watching for stale trades , big spread, big slippage , and quality of filling) , 3rd layer can be for strategy health and performance analytics and drawdown alerts and even strategy divergence in the market from backtesting

answer:

Now you're getting much closer to what could actually become a product. A few months ago, your idea was roughly:

```
Bot alive?  
Exchange connected?
```

That's basically a watchdog.

What you're describing now is more like an **Algo Trading Operations Platform**.

I would structure it exactly the way you're starting to discover it:

```
Layer 1: Operations Health
```

Layer 2: Market Execution Health

Layer 3: Strategy Health

And each layer has its own customer pain.

Layer 1 — Operations Health

This is what you discovered tonight.

VM/VPS

Docker

Freqtrade Engine

Exchange Connectivity

API Availability

Examples:

- VM down
- Container stopped
- Container restarting
- CPU 95%
- RAM exhausted
- Disk full
- Binance API unreachable
- Freqtrade API not responding
- Bot heartbeat missing

This layer is universal.

Every trader has these problems.

Layer 2 — Market Execution Health

This is where your spread × 8 example lives.

This is actually much more interesting.

Not:

Is Binance up?

but:

Is trading quality degrading?

Examples:

- Spread explosion

- Slippage spike
- Order stuck for 10 minutes
- Partial fills
- Excessive cancel/replace loops
- Market latency spike
- Order book imbalance
- Stale candles
- Candle feed delay
- Trade entered 0.8% away from intended price

This is where traders start saying: "Oh shit, this actually happened to me."

Because many traders lose money here without realizing it.

The bot is technically working.

But execution quality is deteriorating.

That's a much more expensive problem.

Layer 3 — Strategy Health

This is the hardest layer.

Because now you're monitoring the behavior of the strategy itself.

Examples:

Expected trade frequency:
15/day

Actual:
1/day

or

Backtest:
57% win rate

Last 30 days:
31% win rate

or

Expected drawdown:
12%

Current:
28%

or

Average holding time:
4 hours

Current:
3 days

Now you're entering:

Strategy Drift Detection

which is extremely valuable.
Not because the bot is broken.
But because:

Market changed.

The strategy still executes perfectly.
Yet its edge disappeared.
Many traders don't notice this until months later.
What's interesting is that these three layers have increasing value:

Layer 1
"Your system is broken."

Layer 2
"Your execution quality is degrading."

Layer 3
"Your edge may be disappearing."

Layer 1 saves infrastructure.
Layer 2 saves trades.
Layer 3 saves strategies.
And the pricing power generally increases as you go up the stack.
The thing that changed tonight is that you discovered Layer 1 is much bigger than you originally thought.
Before Azure you were thinking:

Freqtrade + Binance

Now you've seen:



Every arrow can fail.

Every arrow can be monitored.

And every failure can potentially generate an alert.

That's why spending time running your own bot on a VM is valuable. You're not learning trading. You're building a mental map of where operational failures actually occur. Once you've seen enough failures yourself, the product design becomes much more obvious.